Phase 7 — System Building

ExamGuard AI — Implementation Guide

Camera Streams — Reading Live Video from Real Cameras

What Is This?

So far, you have been working with saved images and video files. Click "run," and the model processes a file on your computer.

But ExamGuard needs to read LIVE video from REAL cameras hanging on the walls of exam halls.

Camera streaming means connecting to physical cameras over a network and reading their video feed in real-time using Python.

How Cameras Send Video Over a Network

The Protocol: RTSP

RTSP (Real Time Streaming Protocol) is how IP cameras share their video over a network. Think of it like a web address, but for video:

```
Web page:  http://google.com
Video feed: rtsp://192.168.1.10:554/stream1
```

Every IP camera has an RTSP URL. When you connect to it, you get a live video stream.

How It Works Step by Step

```
Physical camera on wall
  ↓
Camera connects to network (WiFi or Ethernet cable)
  ↓
Camera gets an IP address (like 192.168.1.10)
  ↓
Camera streams video on port 554 (RTSP default)
  ↓
Your Python code connects to rtsp://192.168.1.10:554/stream
  ↓
OpenCV reads frames from the stream
  ↓
Your AI model processes each frame
```

Types of Cameras

Type	Price	Quality	Use Case
USB Webcam	$20-50	OK	Testing, development
IP Camera	$50-200	Good	Small deployment
PTZ Camera	$200+	Great	Large halls (pan/tilt/zoom)
Phone as Camera	Free	OK	Testing! (use IP Webcam app)

WHY ExamGuard Needs This

From Development to Reality

Development: model.predict("exam_photo.jpg") ← Saved file
Production: model.predict(live_camera_frame) ← Real-time stream

The model is the SAME. The input source changes.

Multi-Camera Setup

Exam Hall Layout:

```
 Camera 1 (front-left)        Camera 2 (front-right)

   ↓                            ↓
[Student] [Student] [Student] [Student] [Student]
[Student] [Student] [Student] [Student] [Student]
[Student] [Student] [Student] [Student] [Student]
[Student] [Student] [Student] [Student] [Student]
   ↑                            ↑

 Camera 3 (back-left)         Camera 4 (back-right)
```

Each camera covers a section of the hall.
Python reads ALL four streams simultaneously.

Reading Camera Streams with Python

Method 1: USB Webcam (Easiest — Start Here)

```
import cv2

# Open webcam (0 = first camera, 1 = second camera)
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("ERROR: Cannot open camera!")
    exit()

while True:
    ret, frame = cap.read()  # Read one frame

    if not ret:
        print("ERROR: Cannot read frame!")
        break

    # Display the frame
    cv2.imshow("Webcam Feed", frame)

    # Press 'q' to quit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

```
cap.release()
cv2.destroyAllWindows()
```

Method 2: IP Camera via RTSP

```
import cv2

# RTSP URL of your IP camera
# Format varies by camera brand:
# Hikvision:  rtsp://username:password@192.168.1.10:554/Streaming/Channels/101
# Dahua:      rtsp://username:password@192.168.1.10:554/cam/realmonitor?channel=1
# Generic:    rtsp://192.168.1.10:554/stream1

rtsp_url = "rtsp://admin:password123@192.168.1.10:554/stream1"

cap = cv2.VideoCapture(rtsp_url)

if not cap.isOpened():
    print("ERROR: Cannot connect to camera!")
    print("Check: Is the camera on? Is the URL correct? Are you on the same network?")
    exit()

while True:
    ret, frame = cap.read()

    if not ret:
        print("Connection lost! Trying to reconnect...")
        cap.release()
        cap = cv2.VideoCapture(rtsp_url)
        continue

    cv2.imshow("IP Camera Feed", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Method 3: Phone as Camera (Free for Testing)

```
Step 1: Install "IP Webcam" app on your Android phone
        (or "EpocCam" for iPhone)

Step 2: Open the app → Start Server

Step 3: It shows a URL like: http://192.168.1.5:8080

Step 4: In Python:
```

```
import cv2

# Use the URL from the phone app
```

```
phone_url = "http://192.168.1.5:8080/video"

cap = cv2.VideoCapture(phone_url)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    cv2.imshow("Phone Camera", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Handling Multiple Cameras

Reading 4 Cameras at Once

```
import cv2
import threading
from queue import Queue

class CameraStream:
    """Read camera in a separate thread so it does not block."""

    def __init__(self, source, name):
        self.name = name
        self.cap = cv2.VideoCapture(source)
        self.frame = None
        self.running = True

        # Start reading in background thread
        self.thread = threading.Thread(target=self._read_frames, daemon=True)
        self.thread.start()

    def _read_frames(self):
        while self.running:
            ret, frame = self.cap.read()
            if ret:
                self.frame = frame
            else:
                print(f"{self.name}: Connection lost! Reconnecting...")
                self.cap.release()
                self.cap = cv2.VideoCapture(self.source)

    def get_frame(self):
        return self.frame

    def stop(self):
```

```
        self.running = False
        self.cap.release()


# Create 4 camera streams
cameras = [
    CameraStream("rtsp://192.168.1.10:554/stream1", "Front-Left"),
    CameraStream("rtsp://192.168.1.11:554/stream1", "Front-Right"),
    CameraStream("rtsp://192.168.1.12:554/stream1", "Back-Left"),
    CameraStream("rtsp://192.168.1.13:554/stream1", "Back-Right"),
]

while True:
    for i, cam in enumerate(cameras):
        frame = cam.get_frame()
        if frame is not None:
            # Resize for display
            small = cv2.resize(frame, (640, 480))
            cv2.imshow(f"Camera {i}: {cam.name}", small)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Clean up
for cam in cameras:
    cam.stop()
cv2.destroyAllWindows()
```

Grid Display (All Cameras in One Window)

```
import numpy as np

def create_camera_grid(frames, grid_size=(2, 2), cell_size=(640, 480)):
    """Arrange multiple camera feeds in a grid."""
    rows, cols = grid_size
    h, w = cell_size

    # Create blank grid
    grid = np.zeros((rows * h, cols * w, 3), dtype=np.uint8)

    for i, frame in enumerate(frames):
        if frame is None:
            continue
        row = i // cols
        col = i % cols
        resized = cv2.resize(frame, (w, h))
        grid[row*h:(row+1)*h, col*w:(col+1)*w] = resized

    return grid

# Use it:
while True:
```

```
    frames = [cam.get_frame() for cam in cameras]
    grid = create_camera_grid(frames)
    cv2.imshow("ExamGuard - All Cameras", grid)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

Handling Common Problems

Problem 1: Camera Disconnects

```
import time

def robust_camera_read(rtsp_url, max_retries=5):
    """Camera reader that automatically reconnects."""
    cap = cv2.VideoCapture(rtsp_url)
    retry_count = 0

    while True:
        ret, frame = cap.read()

        if ret:
            retry_count = 0  # Reset on success
            yield frame
        else:
            retry_count += 1
            print(f"Read failed. Retry {retry_count}/{max_retries}")

            if retry_count >= max_retries:
                print("Reconnecting...")
                cap.release()
                time.sleep(2)  # Wait before reconnecting
                cap = cv2.VideoCapture(rtsp_url)
                retry_count = 0

# Use it:
for frame in robust_camera_read("rtsp://192.168.1.10:554/stream1"):
    result = model.predict(frame)
```

Problem 2: High Latency (Delay)

```
# RTSP streams can buffer frames, causing delay
# Solution: Always grab the LATEST frame, skip old ones

cap = cv2.VideoCapture(rtsp_url)
cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)  # Minimize buffer

# Or: grab multiple times, only process the last one
for _ in range(5):  # Skip 5 buffered frames
    cap.grab()
ret, frame = cap.retrieve()  # Get only the latest
```

Problem 3: Resolution and FPS Settings

```
cap = cv2.VideoCapture(rtsp_url)

# Set resolution
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

# Check actual values (camera may not support your request)
actual_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
actual_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
actual_fps = cap.get(cv2.CAP_PROP_FPS)

print(f"Resolution: {actual_width}x{actual_height}")
print(f"FPS: {actual_fps}")
```

What You Need to Learn

- 1. RTSP protocol basics — how cameras stream video over networks
- 2. OpenCV VideoCapture — the Python tool to read streams
- 3. Threading — reading cameras without blocking your main program
- 4. Error handling — cameras disconnect, network drops, buffers fill up
- 5. Multiple cameras — reading and displaying several feeds at once

Mini Project: Phone Camera to AI Pipeline

Goal

Turn your phone into an IP camera, read the stream in Python, run YOLO on it, and display results.

Steps

Step 1: Setup Phone as Camera

- Install "IP Webcam" app on Android phone
- Connect phone to same WiFi as your computer
- Open app, tap "Start Server"
- Note the URL shown (e.g., http://192.168.1.5:8080)

Step 2: Read and Display

```
import cv2

url = "http://192.168.1.5:8080/video"  # Your phone's URL
cap = cv2.VideoCapture(url)

while True:
    ret, frame = cap.read()
    if not ret:
        print("Cannot read frame!")
        break

    cv2.imshow("Phone Camera", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
        break

cap.release()
cv2.destroyAllWindows()
```

Step 3: Add YOLO Detection

```
from ultralytics import YOLO
import cv2

model = YOLO("yolov8n.pt")
url = "http://192.168.1.5:8080/video"
cap = cv2.VideoCapture(url)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Run YOLO on each frame
    results = model(frame, verbose=False)

    # Draw results on frame
    annotated = results[0].plot()

    # Show
    cv2.imshow("ExamGuard - Live Detection", annotated)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Step 4: Add FPS Counter

```
import time

fps_start = time.time()
frame_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    results = model(frame, verbose=False)
    annotated = results[0].plot()

    # Calculate FPS
    frame_count += 1
    elapsed = time.time() - fps_start
    fps = frame_count / elapsed
```

```
    cv2.putText(annotated, f"FPS: {fps:.1f}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    cv2.imshow("ExamGuard - Live Detection", annotated)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

What You Will Learn

- How to connect to a real camera stream
- How to run AI models on live video
- How to handle real-world issues (latency, disconnects)
- The foundation for ExamGuard's camera system

Key Takeaways

- 1. RTSP is the standard protocol for IP cameras — every camera speaks it
- 2. OpenCV VideoCapture works with URLs — same code for webcam, IP camera, or phone
- 3. Use threading — never read cameras on the main thread (it blocks everything)
- 4. Handle disconnects gracefully — cameras WILL disconnect during a 3-hour exam
- 5. Start with your phone — free, easy, and teaches you everything you need

Edge Computing — Processing Video Near the Camera

What Is This?

Right now, you probably imagine one big powerful computer processing ALL the video from ALL the cameras. That is called "cloud" or "centralized" computing.

Edge computing means putting a small computer RIGHT NEXT TO each camera (or group of cameras) to do SOME processing locally, before sending anything to the central server.

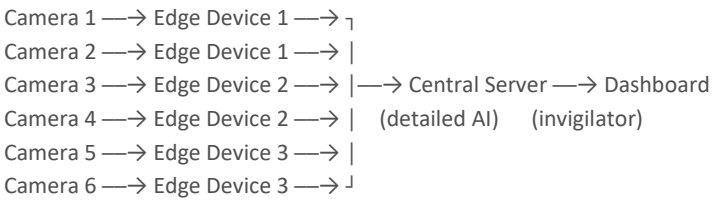
Think of it like this:

WITHOUT Edge Computing:
200 cameras → ALL video → 1 central server → OVERLOADED!
200 cameras × 30fps × 1MB per frame = 6,000 MB per second
That is 6 GB/second of data flowing to ONE server. Impossible.

WITH Edge Computing:
Each camera → small computer next to it → processes locally
→ Only sends SUSPICIOUS frames to central server
200 cameras, but 90% of frames are normal → filtered out locally
Only 10% sent to server = 600 MB/second (manageable!)

How Does It Work?

The Architecture



Edge Device: Runs lightweight YOLO (fast, basic detection)
Central Server: Runs full pipeline (CNN, LSTM, Autoencoder, RL)

What Happens at Each Level

Edge Device (at the camera):

- Runs YOLOv8-nano (fastest, smallest model)
- Detects: "Is there a person? Is there a phone? Any object on desk?"
- Filters: "Is this frame worth analyzing further?"
- Decision: Normal frame → discard. Suspicious frame → send to server.

Central Server (in the control room):

- Receives only suspicious frames (10% of total)
- Runs full analysis: CNN behavior classification, LSTM sequence analysis, autoencoder anomaly detection
- Makes final decision: alert or not
- Sends result to dashboard

WHY ExamGuard Needs This

The Bandwidth Problem

One camera: 1920×1080 resolution, 30 fps
Raw data: 1920 × 1080 × 3 bytes × 30 fps = 186 MB/second
Compressed (H.264): ~2-5 MB/second

20 cameras compressed: 20 × 5 MB = 100 MB/second
Network can handle this (Gigabit Ethernet = 125 MB/s)

200 cameras compressed: 200 × 5 MB = 1,000 MB/second
Network CANNOT handle this. Need 10 Gigabit Ethernet.
And one server cannot process all of it.

The Processing Problem

Full AI pipeline per frame: ~25ms
200 cameras × 30 fps = 6,000 frames/second
6,000 × 25ms = 150,000ms of compute per second
Need 150 parallel GPU cores to keep up!

WITH edge computing:
Edge filters out 90% → only 600 frames/second reach server
600 × 25ms = 15,000ms → Need only 15 GPU cores. MUCH cheaper!

The Latency Problem

Without edge:
Camera → Network (50ms) → Server queue (200ms) → Process (25ms) → Alert
Total: 275ms delay. Nearly a third of a second late.

With edge:
Camera → Edge device (2ms) → Quick check (5ms)
If suspicious → Server (50ms) → Process (25ms) → Alert
Total for normal frames: 7ms (no delay)
Total for suspicious: 82ms (much faster than 275ms)

Edge Computing Hardware

Option 1: NVIDIA Jetson Nano ($150-200)

What: A tiny computer with a GPU, size of a credit card
GPU: 128 CUDA cores
RAM: 4 GB
Power: 5-10 watts (plugs into wall outlet)

Can run: YOLOv8-nano at 15-25 fps
Perfect for: 1-2 cameras per device

Option 2: NVIDIA Jetson Orin Nano ($500)

What: Upgraded version, much more powerful
GPU: 1024 CUDA cores
RAM: 8 GB

Power: 7-15 watts

Can run: YOLOv8-small at 30+ fps
Perfect for: 4-6 cameras per device

Option 3: Raspberry Pi 5 + Coral TPU ($100 total)

What: Tiny general-purpose computer + Google's AI accelerator
GPU: None (uses TPU instead)
RAM: 4-8 GB
Power: 5 watts

Can run: Lightweight models at 10-15 fps
Perfect for: Simple detection, 1 camera per device
Cheaper but less powerful than Jetson

Option 4: Intel NUC with GPU ($300-600)

What: Mini PC, more powerful than Jetson
GPU: Intel integrated or small discrete GPU
RAM: 8-16 GB

Can run: YOLOv8-small/medium at 30+ fps
Perfect for: 4-8 cameras per device

Making Models Lightweight for Edge

Model Compression Techniques

1. Use Smaller Model Variants

```
from ultralytics import YOLO

# Full model (for server):
server_model = YOLO("yolov8m.pt")  # 25.9M parameters, accurate

# Edge model (for Jetson):
edge_model = YOLO("yolov8n.pt")    # 3.2M parameters, fast!

# Comparison:
# yolov8n: 3.2M params, 8.7 GFLOPs  → Edge
# yolov8s: 11.2M params, 28.6 GFLOPs → Good edge device
# yolov8m: 25.9M params, 78.9 GFLOPs → Server
# yolov8l: 43.7M params, 165 GFLOPs  → Powerful server
```

2. Export for Edge (TensorRT)

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

# Export optimized for Jetson
model.export(
   format="engine",      # TensorRT format
   half=True,          # FP16 (half precision — 2x faster)
```

```
    device=0            # Optimize for this specific GPU
)
# Creates: yolov8n.engine (runs 2-3x faster than .pt on Jetson)
```

3. Reduce Input Resolution

```
# Instead of processing 1920x1080 (HD), resize to 640x480
# Faster processing, still good enough for detection

import cv2

frame = camera.read()  # 1920x1080
small_frame = cv2.resize(frame, (640, 480))  # Much faster to process
results = model(small_frame)
```

Edge Device Code Example

What Runs on the Edge Device

```
"""
ExamGuard Edge Device Script
Runs on NVIDIA Jetson at each camera location.
Only sends suspicious frames to central server.
"""

import cv2
import requests
import json
import time
from ultralytics import YOLO

# Configuration
CAMERA_URL = "rtsp://192.168.1.10:554/stream1"
SERVER_URL = "http://central-server:8000/api/frame"
CAMERA_ID = "hall1_cam1"

# Load lightweight model
model = YOLO("yolov8n.engine")  # TensorRT optimized

# Connect to camera
cap = cv2.VideoCapture(CAMERA_URL)

suspicious_classes = ['cell phone', 'book', 'paper']  # Objects to flag
frame_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        continue

    frame_count += 1

    # Only process every 3rd frame (10 fps is enough)
```

```
if frame_count % 3 != 0:
    continue

# Run lightweight detection
results = model(frame, verbose=False)

# Check if anything suspicious was detected
is_suspicious = False
detections = []

for box in results[0].boxes:
    class_name = results[0].names[int(box.cls)]
    confidence = float(box.conf)

    if class_name in suspicious_classes and confidence > 0.5:
        is_suspicious = True
        detections.append({
            'class': class_name,
            'confidence': confidence,
            'bbox': box.xyxy[0].tolist()
        })

if is_suspicious:
    # ONLY send suspicious frames to server
    _, img_encoded = cv2.imencode('.jpg', frame)

    payload = {
        'camera_id': CAMERA_ID,
        'timestamp': time.time(),
        'detections': json.dumps(detections)
    }

    try:
        requests.post(
            SERVER_URL,
            files={'frame': img_encoded.tobytes()},
            data=payload,
            timeout=2
        )
        print(f"Sent suspicious frame: {detections}")
    except Exception as e:
        print(f"Failed to send: {e}")
else:
    # Normal frame — discard, do not send
    pass
```

What You Need to Learn

- 1. Edge vs Cloud computing — when to process locally vs remotely
- 2. NVIDIA Jetson platform — the go-to hardware for edge AI
- 3. Model compression — making models small enough for edge devices

- 4. Network architecture — how edge devices communicate with the server
- 5. Power and cost planning — how many edge devices for your deployment

Mini Project: Run YOLO on a Raspberry Pi or Jetson

If You Have a Jetson Nano

Step 1: Flash JetPack OS

- Download JetPack from NVIDIA website
- Flash to microSD card using Etcher
- Boot the Jetson, complete setup

Step 2: Install Dependencies

```
# On Jetson terminal:
pip install ultralytics opencv-python
```

Step 3: Run YOLO

```
from ultralytics import YOLO
import cv2

model = YOLO("yolov8n.pt")

# Use USB webcam connected to Jetson
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    results = model(frame, verbose=False)
    annotated = results[0].plot()

    cv2.imshow("Jetson Detection", annotated)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

Step 4: Measure FPS

```
import time

start = time.time()
count = 0

while count < 100:
    ret, frame = cap.read()
    results = model(frame, verbose=False)
    count += 1

elapsed = time.time() - start
print(f"FPS on Jetson: {count/elapsed:.1f}")
```

If You Do NOT Have a Jetson (Simulate It)

You can simulate edge computing on your regular computer:

```
"""
Simulate edge computing:
- Run lightweight YOLO (pretend it is on edge device)
- Only forward suspicious frames (pretend sending to server)
- Run full pipeline on suspicious frames only (pretend this is the server)
"""

lightweight_model = YOLO("yolov8n.pt")   # "Edge" model
full_model = YOLO("yolov8m.pt")          # "Server" model

frames_processed_edge = 0
frames_sent_to_server = 0

for frame in video_frames:
    frames_processed_edge += 1

    # Edge processing (fast check)
    results = lightweight_model(frame, verbose=False)
    has_suspicious = any(box.conf > 0.5 for box in results[0].boxes)

    if has_suspicious:
        # "Send to server" — run full pipeline
        frames_sent_to_server += 1
        full_results = full_model(frame, verbose=False)
        # Process full_results...

print(f"Edge processed: {frames_processed_edge} frames")
print(f"Sent to server: {frames_sent_to_server} frames")
print(f"Filtered out: {(1 - frames_sent_to_server/frames_processed_edge)*100:.0f}%")
```

Key Takeaways

- 1. Edge computing processes data near the source — reduces network load and latency
- 2. 90% of frames are normal — edge devices filter them, server only handles the interesting 10%
- 3. NVIDIA Jetson is the go-to edge device — small, cheap, GPU-equipped
- 4. Use lightweight models at the edge — YOLOv8-nano, TensorRT optimized
- 5. Edge + Server = best of both worlds — fast local filtering, deep central analysis

GPU Programming — Making AI 100x Faster

What Is This?

Your computer has two main processors:

CPU (Central Processing Unit) — The "brain." Does one thing at a time, but very smartly. Good at complex tasks like running your operating system, web browser, Word document.

GPU (Graphics Processing Unit) — Originally for video games. Does THOUSANDS of simple things at the same time. Good at math on lots of numbers simultaneously.

Deep learning is basically doing the SAME math on MILLIONS of numbers. That is EXACTLY what GPUs are designed for.

CPU: 1 chef making 1 dish at a time (but a really good chef)
  Makes 8 dishes per hour (8 cores)

GPU: 5,000 simple cooks, each makes 1 dish at a time
  Makes 5,000 dishes per hour (5,000 CUDA cores)

Deep learning needs 5,000 simple dishes, not 8 complex ones.
GPU wins by a MASSIVE margin.

WHY ExamGuard Needs This

Speed Comparison

Task: Run YOLO on one image (detect objects)

CPU (Intel i7): ~150ms per image = ~7 fps
GPU (RTX 3060): ~8ms per image = ~125 fps
GPU (RTX 4090): ~3ms per image = ~333 fps

CPU: Can barely handle 1 camera at 7 fps (choppy, delayed)
GPU: Can handle 4+ cameras at 30 fps (smooth, real-time)

ExamGuard is IMPOSSIBLE Without GPU

Full pipeline per frame: ~25ms on GPU, ~800ms on CPU

On CPU: 1 camera at 1.2 fps (2.5 seconds behind real-time) — USELESS
On GPU: 4 cameras at 30 fps (real-time) — PERFECT

Scaling

Small exam (1 hall, 4 cameras):
  1x NVIDIA RTX 3080 = enough

Medium exam (5 halls, 20 cameras):
  2x NVIDIA RTX 4080 = enough

Large exam (50 halls, 200 cameras):
  Edge computing + 4x NVIDIA A100 server GPUs
  Or: 10x RTX 4090 consumer GPUs

How GPU Programming Works

CUDA — NVIDIA's GPU Programming Platform

CUDA (Compute Unified Device Architecture) lets you run code on NVIDIA GPUs.

You do NOT need to write CUDA code directly. PyTorch does it for you. But you need to understand the basics.

```
Your Python code
  ↓
PyTorch (.to('cuda'))
  ↓
CUDA translates to GPU instructions
  ↓
GPU executes thousands of operations in parallel
  ↓
Results sent back to CPU
```

The Key Concept: Moving Data to GPU

```
import torch

# Data on CPU (default)
tensor_cpu = torch.randn(1000, 1000)
print(tensor_cpu.device)  # cpu

# Move data to GPU
tensor_gpu = tensor_cpu.to('cuda')
print(tensor_gpu.device)  # cuda:0

# Or create directly on GPU
tensor_gpu = torch.randn(1000, 1000, device='cuda')
```

IMPORTANT: Model AND Data Must Be on Same Device

```
model = MyModel()

# WRONG: Model on CPU, data on GPU
model = model.cpu()
data = data.cuda()
output = model(data)  # ERROR! Device mismatch!

# CORRECT: Both on GPU
model = model.cuda()
data = data.cuda()
output = model(data)  # Works!

# Or use .to() for flexibility
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)
data = data.to(device)
output = model(data)
```

Setting Up GPU for Deep Learning

Step 1: Check Your GPU

```
import torch

# Do you have a GPU?
print(f"CUDA available: {torch.cuda.is_available()}")

if torch.cuda.is_available():
    print(f"GPU name: {torch.cuda.get_device_name(0)}")
    print(f"GPU memory: {torch.cuda.get_device_properties(0).total_mem / 1e9:.1f} GB")
    print(f"CUDA version: {torch.version.cuda}")
    print(f"Number of GPUs: {torch.cuda.device_count()}")
else:
    print("No GPU found. Install NVIDIA drivers and CUDA toolkit.")
```

Step 2: Install the Right Software

```
# 1. NVIDIA GPU Driver (from nvidia.com — match your GPU model)
# 2. CUDA Toolkit (from developer.nvidia.com/cuda-downloads)
# 3. cuDNN (from developer.nvidia.com/cudnn — for deep learning)
# 4. PyTorch with CUDA support:

pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
# Replace cu121 with your CUDA version (cu118, cu121, etc.)
```

Step 3: Verify Installation

```
import torch

# This should print True
print(torch.cuda.is_available())

# Quick GPU test
a = torch.randn(1000, 1000, device='cuda')
b = torch.randn(1000, 1000, device='cuda')
c = torch.matmul(a, b)  # Matrix multiplication on GPU
print(f"Result shape: {c.shape}, device: {c.device}")
print("GPU is working!")
```

Using GPU in Your ExamGuard Code

Training a Model on GPU

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader

# Pick device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Create model and move to GPU
```

```
model = YourModel()
model = model.to(device)

# Loss and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Training loop
for epoch in range(10):
    for images, labels in train_loader:
        # Move data to GPU
        images = images.to(device)
        labels = labels.to(device)

        # Forward pass (runs on GPU)
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Backward pass (runs on GPU)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch}: Loss = {loss.item():.4f}")
```

Running YOLO with GPU

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

# Run on GPU (YOLO handles it automatically)
results = model("test_image.jpg", device=0)  # device=0 means first GPU

# Or for live camera:
results = model(frame, device=0, verbose=False)
```

Memory Management

```
# Check GPU memory usage
print(f"Allocated: {torch.cuda.memory_allocated(0) / 1e9:.2f} GB")
print(f"Cached: {torch.cuda.memory_reserved(0) / 1e9:.2f} GB")

# Clear unused memory
torch.cuda.empty_cache()

# If you run out of memory, try:
# 1. Reduce batch size
# 2. Use smaller model
# 3. Use mixed precision (FP16)
# 4. Use gradient checkpointing
```

Mixed Precision Training (Faster + Less Memory)

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()

for images, labels in train_loader:
    images = images.to(device)
    labels = labels.to(device)

    optimizer.zero_grad()

    # Mixed precision: use FP16 where possible (2x faster, half memory)
    with autocast():
        outputs = model(images)
        loss = criterion(outputs, labels)

    # Scale gradients to prevent underflow in FP16
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

GPU Memory: The Main Constraint

How Much Memory Do You Need?

Model	GPU Memory Needed
YOLOv8-nano	~0.5 GB
YOLOv8-small	~1.0 GB
YOLOv8-medium	~2.0 GB
ResNet18	~0.5 GB
ResNet50	~1.5 GB
LSTM (sequence model)	~0.3 GB
Autoencoder	~0.5 GB
Full ExamGuard pipeline	~3-5 GB total

Common GPUs:
RTX 3060: 12 GB → Plenty for ExamGuard
RTX 3080: 10 GB → Good
RTX 4090: 24 GB → Overkill (but fast!)
A100: 40/80 GB → Server GPU, massive

What If You Run Out of Memory?

Error: "CUDA out of memory"

Solutions:
1. Reduce batch size (batch_size=32 → batch_size=8)
2. Use smaller model (yolov8m → yolov8n)
3. Reduce input resolution (640x640 → 320x320)
4. Use mixed precision (FP16 instead of FP32)

5. Free memory: torch.cuda.empty_cache()
6. Use gradient checkpointing (trades speed for memory)

Multiple GPUs

If You Have Multiple GPUs

```
# Check number of GPUs
print(f"GPUs available: {torch.cuda.device_count()}")
for i in range(torch.cuda.device_count()):
    print(f"  GPU {i}: {torch.cuda.get_device_name(i)}")

# Simple approach: Different models on different GPUs
yolo_model = YOLO("yolov8n.pt").to('cuda:0')   # GPU 0
cnn_model = CNN().to('cuda:1')                 # GPU 1

# Or: DataParallel (same model on multiple GPUs, bigger batches)
model = nn.DataParallel(model)  # Automatically splits batch across GPUs
```

Mini Project: CPU vs GPU Speed Comparison

Goal

Measure how much faster GPU is compared to CPU for a real model.

```
import torch
import torchvision.models as models
import time

# Load a pre-trained model
model = models.resnet18(pretrained=True)
model.eval()

# Create fake input (batch of 32 images, 3 channels, 224x224)
input_data = torch.randn(32, 3, 224, 224)

# —— CPU Benchmark ——
model_cpu = model.cpu()
input_cpu = input_data.cpu()

# Warm up
for _ in range(5):
    with torch.no_grad():
        model_cpu(input_cpu)

# Measure
start = time.time()
for _ in range(100):
    with torch.no_grad():
        output = model_cpu(input_cpu)
cpu_time = (time.time() - start) / 100 * 1000  # ms per batch
print(f"CPU: {cpu_time:.1f}ms per batch of 32")
print(f"CPU: {cpu_time/32:.1f}ms per image")
```

```
print(f"CPU: {32000/cpu_time:.0f} images/second")

# —— GPU Benchmark ——
if torch.cuda.is_available():
    model_gpu = model.cuda()
    input_gpu = input_data.cuda()

    # Warm up
    for _ in range(10):
        with torch.no_grad():
            model_gpu(input_gpu)
    torch.cuda.synchronize()

    # Measure
    start = time.time()
    for _ in range(100):
        with torch.no_grad():
            output = model_gpu(input_gpu)
    torch.cuda.synchronize()
    gpu_time = (time.time() - start) / 100 * 1000

    print(f"\nGPU: {gpu_time:.1f}ms per batch of 32")
    print(f"GPU: {gpu_time/32:.1f}ms per image")
    print(f"GPU: {32000/gpu_time:.0f} images/second")

    print(f"\nGPU is {cpu_time/gpu_time:.1f}x FASTER than CPU!")
else:
    print("\nNo GPU available. Install CUDA to compare.")
```

Expected Results (approximate):

```
CPU: 450.0ms per batch of 32
CPU: 14.1ms per image
CPU: 71 images/second

GPU: 12.0ms per batch of 32
GPU: 0.4ms per image
GPU: 2667 images/second

GPU is 37.5x FASTER than CPU!
```

Key Takeaways

- 1. GPU is 10-100x faster than CPU for deep learning tasks
- 2. ExamGuard is impossible without GPU — cannot process real-time video on CPU
- 3. Move both model AND data to GPU — they must be on the same device
- 4. GPU memory is the main constraint — monitor usage, reduce if needed
- 5. Use mixed precision (FP16) for 2x speed and half memory usage
- 6. Start with 1 GPU — it is enough for development and small deployments
- 7. Scale with multiple GPUs or edge computing for large deployments

Dashboard & Web UI — The Invigilator's Screen

What Is This?

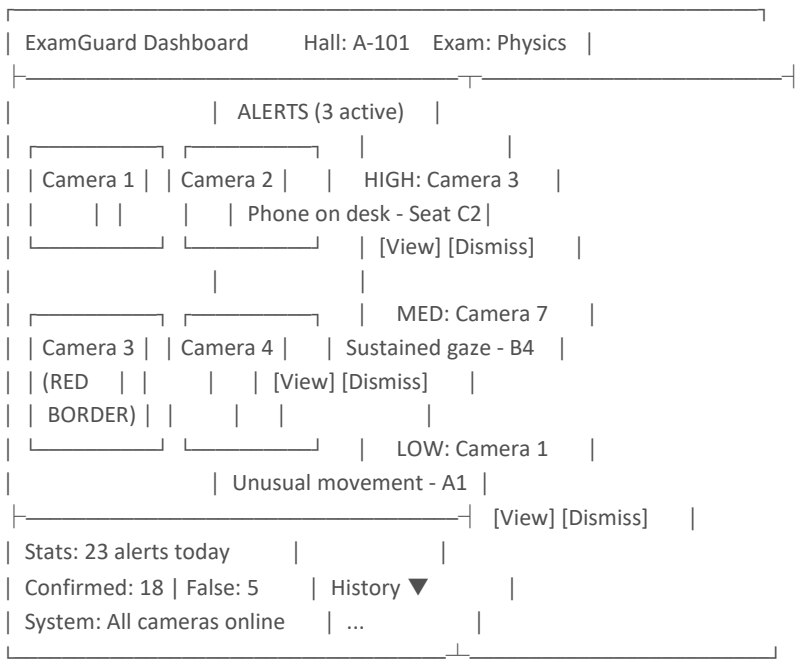
You have built all the AI models. They can detect phones, track gaze, spot anomalies, and make decisions. But right now, the output is just text in a terminal:

```
[14:23:05] Camera 3: Phone detected, confidence 0.92
[14:23:07] Camera 7: Suspicious gaze pattern, student seat B4
[14:23:12] Camera 1: Anomaly detected, reconstruction error 0.45
```

An invigilator cannot stare at terminal text during a 3-hour exam. They need a VISUAL dashboard.

The dashboard is the HUMAN INTERFACE — where all AI outputs become useful, actionable information on a screen.

What the Dashboard Looks Like



WHY ExamGuard Needs This

AI Is Not the Final Decision Maker

ExamGuard is a TOOL for invigilators, not a replacement:

```
AI detects something suspicious
  ↓
Dashboard shows alert with evidence
  ↓
Invigilator reviews the evidence
  ↓
Invigilator decides: real cheating or false alarm
  ↓
Invigilator's decision is recorded
```

↓
AI learns from this feedback (gets better over time)

What the Invigilator Needs

- 1. Live camera feeds — see all cameras at once
- 2. Alert notifications — know immediately when something is flagged
- 3. Priority levels — high alerts first, low alerts later
- 4. Evidence clips — short video showing what triggered the alert
- 5. One-click actions — confirm cheating, dismiss false alarm, request closer look
- 6. Statistics — how many alerts, accuracy rate, active cameras

Technology Stack

Backend: FastAPI (Python)

FastAPI handles:
- Receiving alerts from AI models
- Serving camera feeds to the browser
- Managing the database
- Sending real-time updates to the dashboard

Why FastAPI?
- Written in Python (same as your AI code!)
- FAST (async support)
- Easy to learn
- Great documentation

Frontend: HTML + CSS + JavaScript

The browser displays:
- Camera grid
- Alert panel
- Statistics
- Interactive buttons

You do NOT need React or Vue for the first version.
Plain HTML + CSS + JavaScript is enough.

Real-Time Updates: WebSockets

Normal web: Browser asks server "any updates?" every 5 seconds (polling)
WebSocket: Server PUSHES updates to browser instantly (real-time)

For ExamGuard: When AI detects something, dashboard shows it IMMEDIATELY.

Building the Dashboard Step by Step

Step 1: Basic Flask Web Server

```
from flask import Flask, render_template, Response
import cv2

app = Flask(__name__)
```

```
@app.route('/')
def index():
    return render_template('dashboard.html')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Step 2: HTML Dashboard Template

Create `templates/dashboard.html`:

```
<!DOCTYPE html>
<html>
<head>
    <title>ExamGuard Dashboard</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 20px;
            background: #1a1a2e;
            color: white;
        }
        .header {
            text-align: center;
            padding: 10px;
            background: #16213e;
            border-radius: 10px;
            margin-bottom: 20px;
        }
        .container {
            display: flex;
            gap: 20px;
        }
        .camera-grid {
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 10px;
            flex: 2;
        }
        .camera-feed {
            background: #0f3460;
            border-radius: 10px;
            padding: 10px;
            text-align: center;
        }
        .camera-feed img {
            width: 100%;
            border-radius: 5px;
        }
        .camera-feed.alert {
```

```
            border: 3px solid red;
            animation: pulse 1s infinite;
        }
        @keyframes pulse {
            0% { border-color: red; }
            50% { border-color: transparent; }
            100% { border-color: red; }
        }
        .alert-panel {
            flex: 1;
            background: #16213e;
            border-radius: 10px;
            padding: 15px;
        }
        .alert-item {
            background: #0f3460;
            border-radius: 8px;
            padding: 10px;
            margin-bottom: 10px;
        }
        .alert-high { border-left: 4px solid red; }
        .alert-medium { border-left: 4px solid orange; }
        .alert-low { border-left: 4px solid green; }
        .btn {
            padding: 5px 10px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
            margin: 2px;
        }
        .btn-confirm { background: #e94560; color: white; }
        .btn-dismiss { background: #533483; color: white; }
    </style>
</head>
<body>
    <div class="header">
        <h1>ExamGuard Dashboard</h1>
        <p>Hall: A-101 | Exam: Physics | Time: <span id="clock"></span></p>
    </div>

    <div class="container">
        <div class="camera-grid">
            <div class="camera-feed" id="cam1">
                <h3>Camera 1 - Front Left</h3>
                <img src="/video_feed/1" alt="Camera 1">
            </div>
            <div class="camera-feed" id="cam2">
                <h3>Camera 2 - Front Right</h3>
                <img src="/video_feed/2" alt="Camera 2">
            </div>
            <div class="camera-feed" id="cam3">
```

```
            <h3>Camera 3 - Back Left</h3>
            <img src="/video_feed/3" alt="Camera 3">
        </div>
        <div class="camera-feed" id="cam4">
            <h3>Camera 4 - Back Right</h3>
            <img src="/video_feed/4" alt="Camera 4">
        </div>
    </div>

    <div class="alert-panel">
        <h2>Alerts</h2>
        <div id="alerts">
            <!-- Alerts will be added here dynamically -->
        </div>
        <hr>
        <h3>Statistics</h3>
        <p>Total alerts: <span id="total-alerts">0</span></p>
        <p>Confirmed: <span id="confirmed">0</span></p>
        <p>Dismissed: <span id="dismissed">0</span></p>
    </div>
</div>

<script>
    // Update clock
    setInterval(() => {
        document.getElementById('clock').textContent =
            new Date().toLocaleTimeString();
    }, 1000);

    // Connect to WebSocket for real-time alerts
    const ws = new WebSocket('ws://localhost:5000/ws');

    ws.onmessage = function(event) {
        const alert = JSON.parse(event.data);
        addAlert(alert);
    };

    function addAlert(alert) {
        const alertsDiv = document.getElementById('alerts');
        const alertHtml = `
            <div class="alert-item alert-${alert.priority}">
                <strong>${alert.priority.toUpperCase()}</strong>: Camera ${alert.camera_id}<br>
                ${alert.description}<br>
                Seat: ${alert.seat} | Confidence: ${(alert.confidence * 100).toFixed(0)}%<br>
                <button class="btn btn-confirm" onclick="confirmAlert(${alert.id})">Confirm</button>
                <button class="btn btn-dismiss" onclick="dismissAlert(${alert.id})">Dismiss</button>
            </div>
        `;
        alertsDiv.insertAdjacentHTML('afterbegin', alertHtml);

        // Highlight the camera feed
```

```
            document.getElementById('cam' + alert.camera_id).classList.add('alert');

            // Update counter
            const total = document.getElementById('total-alerts');
            total.textContent = parseInt(total.textContent) + 1;
        }

        function confirmAlert(id) {
            fetch('/api/alert/' + id + '/confirm', {method: 'POST'});
            const confirmed = document.getElementById('confirmed');
            confirmed.textContent = parseInt(confirmed.textContent) + 1;
        }

        function dismissAlert(id) {
            fetch('/api/alert/' + id + '/dismiss', {method: 'POST'});
            const dismissed = document.getElementById('dismissed');
            dismissed.textContent = parseInt(dismissed.textContent) + 1;
        }
    </script>
</body>
</html>
```

Step 3: Video Streaming from Camera

```
from flask import Flask, render_template, Response
import cv2

app = Flask(__name__)

def generate_frames(camera_id):
    """Stream video frames as MJPEG."""
    # In real system, connect to RTSP camera
    # For testing, use webcam
    cap = cv2.VideoCapture(0)

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        # Encode frame as JPEG
        _, buffer = cv2.imencode('.jpg', frame)
        frame_bytes = buffer.tobytes()

        # Yield as multipart response (MJPEG stream)
        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + frame_bytes + b'\r\n')

@app.route('/video_feed/<int:camera_id>')
def video_feed(camera_id):
    return Response(
        generate_frames(camera_id),
```

```
        mimetype='multipart/x-mixed-replace; boundary=frame'
    )

@app.route('/')
def index():
    return render_template('dashboard.html')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step 4: WebSocket for Real-Time Alerts

```
from flask import Flask
from flask_socketio import SocketIO, emit
import json

app = Flask(__name__)
socketio = SocketIO(app)

def send_alert(alert_data):
    """Called by AI system when something suspicious is detected."""
    socketio.emit('new_alert', json.dumps(alert_data))

# Example: AI detects something
alert = {
    'id': 1,
    'camera_id': 3,
    'priority': 'high',
    'description': 'Phone detected on desk',
    'seat': 'C2',
    'confidence': 0.92,
    'timestamp': '14:23:05'
}
send_alert(alert)
```

What You Need to Learn

Web Development Basics (Just Enough)

- 1. HTML — Structure of the page (headings, divs, images)
- 2. CSS — Styling (colors, layout, animations)
- 3. JavaScript — Interactivity (buttons, real-time updates)
- 4. Flask — Python web framework (serves pages, handles API calls)
- 5. WebSockets — Real-time two-way communication

You Do NOT Need to Be a Web Developer

You only need enough to build a functional dashboard. It does not need to look like a professional website. FUNCTION over FORM.

Mini Project: Simple Dashboard with Webcam + Fake Alerts

Goal

Build a basic dashboard that shows your webcam feed and displays fake alerts.

Step 1: Install Flask

```
pip install flask flask-socketio
```

Step 2: Create Project Structure

```
dashboard_project/
├── app.py
├── templates/
│   └── dashboard.html
└── static/
    └── style.css
```

Step 3: Create app.py

```
from flask import Flask, render_template, Response, jsonify
import cv2
import time
import threading
import random

app = Flask(__name__)

# Store alerts
alerts = []
alert_id = 0

def generate_frames():
    cap = cv2.VideoCapture(0)
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        _, buffer = cv2.imencode('.jpg', frame)
        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + buffer.tobytes() + b'\r\n')

@app.route('/')
def index():
    return render_template('dashboard.html')

@app.route('/video_feed')
def video_feed():
    return Response(generate_frames(),
                    mimetype='multipart/x-mixed-replace; boundary=frame')

@app.route('/api/alerts')
def get_alerts():
    return jsonify(alerts[-10:])  # Last 10 alerts
```

```
# Simulate fake alerts every 10 seconds (for testing)
def fake_alert_generator():
    global alert_id
    descriptions = [
        "Phone detected on desk",
        "Sustained gaze at neighbor",
        "Unusual hand movement",
        "Possible note passing",
        "Student looking at another paper"
    ]
    priorities = ["high", "medium", "low"]
    seats = ["A1", "B3", "C2", "D5", "E4"]

    while True:
        time.sleep(random.randint(5, 15))
        alert_id += 1
        alert = {
            'id': alert_id,
            'description': random.choice(descriptions),
            'priority': random.choice(priorities),
            'seat': random.choice(seats),
            'confidence': round(random.uniform(0.6, 0.99), 2),
            'time': time.strftime("%H:%M:%S")
        }
        alerts.append(alert)
        print(f"New alert: {alert}")

# Start fake alerts in background
threading.Thread(target=fake_alert_generator, daemon=True).start()

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Step 4: Run It

```
python app.py
# Open browser: http://localhost:5000
```

You should see your webcam feed and alerts appearing every few seconds. This is the FOUNDATION of ExamGuard's dashboard.

Key Takeaways

- 1. The dashboard is where AI becomes useful — without it, the AI is just printing text
- 2. Flask + HTML/CSS/JS is enough — you do not need React or complex frameworks to start
- 3. WebSockets provide real-time updates — alerts appear instantly, not on page refresh
- 4. Human-in-the-loop is the design — AI flags, dashboard presents, human decides
- 5. Start simple — webcam feed + fake alerts. Add complexity gradually.
- 6. The dashboard IS the product — from the invigilator's perspective, the dashboard IS ExamGuard

Database — Storing Everything ExamGuard Sees and Does

What Is This?

Every alert, every detection, every invigilator action, every video clip — it all needs to be SAVED somewhere. That somewhere is a database.

Think of a database like a super-organized spreadsheet that:

- Can hold millions of rows
- Multiple programs can read/write at the same time
- Data is safe even if the power goes out
- You can search through everything instantly

WHY ExamGuard Needs a Database

Reason 1: Evidence After the Exam

After exam ends, the review committee asks:
"Show us all alerts from Hall A-101, Camera 3, between 2:00 PM and 2:30 PM"

Without database: "Uh... the terminal output scrolled away. It is gone."
With database: 3 results found. Here are the video clips, timestamps,
seat numbers, confidence levels, and invigilator actions.

Reason 2: Improving the AI

After 100 exams, you want to know:
- What is the false alarm rate?
- Which camera angles produce the most alerts?
- What types of cheating are most common?
- Is the system improving over time?

The database has ALL this data. Run a query, get the answer.

Reason 3: Legal Protection

If a student is accused of cheating based on AI detection, you need:

- Exact timestamp of detection
- Video evidence
- Confidence level
- Which AI model flagged it
- The invigilator's decision
- Complete audit trail

Without proper records, the system has no credibility.

Reason 4: Real-Time Operations

During the exam, the dashboard needs to:

- Show current alert count
- List recent alerts
- Track which alerts have been reviewed

- Show camera status (online/offline)

All of this is read from the database in real-time.

What to Store

Table 1: Exams

id | exam_name | hall_id | date | start_time | end_time | status
1 | Physics Final | A-101 | 2026-03-15 | 14:00 | 17:00 | completed
2 | Math Midterm | B-203 | 2026-03-16 | 09:00 | 11:00 | in_progress

Table 2: Cameras

id | camera_name | hall_id | rtsp_url | status | position
1 | Front-Left | A-101 | rtsp://192.168.1.10:554/s1 | online | front-left
2 | Front-Right | A-101 | rtsp://192.168.1.11:554/s1 | online | front-right
3 | Back-Left | A-101 | rtsp://192.168.1.12:554/s1 | offline | back-left

Table 3: Alerts (Most Important)

id | exam_id | camera_id | timestamp | alert_type | seat | confidence | priority | evidence_path | status
1 | 1 | 3 | 2026-03-15 14:23:05 | phone_detected | C2 | 0.92 | high | /evidence/alert_001.mp4 | confirmed
2 | 1 | 7 | 2026-03-15 14:23:07 | sustained_gaze | B4 | 0.78 | medium | /evidence/alert_002.mp4 | dismissed
3 | 1 | 1 | 2026-03-15 14:23:12 | anomaly | A1 | 0.65 | low | /evidence/alert_003.mp4 | pending

Table 4: Invigilator Actions

id | alert_id | invigilator_id | action | timestamp | notes
1 | 1 | inv_001 | confirmed | 2026-03-15 14:23:30 | "Student had phone under paper"
2 | 2 | inv_001 | dismissed | 2026-03-15 14:24:00 | "Student was looking at clock"

Table 5: Students (Optional)

id | name | seat | exam_id | photo_path
S001 | Ahmed Khan | A1 | 1 | /students/ahmed.jpg
S002 | Sara Ali | B4 | 1 | /students/sara.jpg

Technology Choices

PostgreSQL — Main Database

What: A powerful, free, open-source relational database
Why: Reliable, handles millions of rows, great for structured data
Use for: Alerts, exams, cameras, students, invigilator actions

Redis — Real-Time Cache

What: Super-fast in-memory database
Why: When dashboard needs INSTANT data (current alerts, camera status)
Use for: Current active alerts, camera heartbeats, live statistics
Speed: Reads in < 1 millisecond (PostgreSQL takes 5-20ms)

Cloud/Local Storage — Video Clips

What: File storage for video evidence
Why: Video files are too large for a database
Use for: Alert evidence clips, full exam recordings
Options: Local disk, NAS, AWS S3, Google Cloud Storage

Setting Up PostgreSQL

Step 1: Install

```
# Windows: Download from postgresql.org
# Linux:
sudo apt install postgresql postgresql-contrib
# Mac:
brew install postgresql
```

Step 2: Connect with Python

```
pip install psycopg2-binary  # PostgreSQL adapter for Python
# or
pip install sqlalchemy       # ORM (easier to use)
```

Step 3: Create Tables

```
import psycopg2

# Connect to PostgreSQL
conn = psycopg2.connect(
   host="localhost",
   database="examguard",
   user="postgres",
   password="your_password"
)
cursor = conn.cursor()

# Create alerts table
cursor.execute("""
   CREATE TABLE IF NOT EXISTS alerts (
      id SERIAL PRIMARY KEY,
      exam_id INTEGER NOT NULL,
      camera_id INTEGER NOT NULL,
      timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
      alert_type VARCHAR(50) NOT NULL,
      seat VARCHAR(10),
      confidence FLOAT,
      priority VARCHAR(10),
      evidence_path VARCHAR(255),
      status VARCHAR(20) DEFAULT 'pending'
   )
""")

# Create invigilator_actions table
cursor.execute("""
```

```
    CREATE TABLE IF NOT EXISTS invigilator_actions (
        id SERIAL PRIMARY KEY,
        alert_id INTEGER REFERENCES alerts(id),
        invigilator_id VARCHAR(50),
        action VARCHAR(20),
        timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        notes TEXT
    )
"""")

conn.commit()
print("Tables created!")
```

Step 4: Insert an Alert

```
def save_alert(exam_id, camera_id, alert_type, seat, confidence, priority, evidence_path):
    """Save a new alert to the database."""
    cursor.execute("""
        INSERT INTO alerts (exam_id, camera_id, alert_type, seat, confidence, priority, evidence_path)
        VALUES (%s, %s, %s, %s, %s, %s, %s)
        RETURNING id
    """, (exam_id, camera_id, alert_type, seat, confidence, priority, evidence_path))

    alert_id = cursor.fetchone()[0]
    conn.commit()
    print(f"Alert saved with ID: {alert_id}")
    return alert_id

# When AI detects something:
save_alert(
    exam_id=1,
    camera_id=3,
    alert_type="phone_detected",
    seat="C2",
    confidence=0.92,
    priority="high",
    evidence_path="/evidence/alert_001.mp4"
)
```

Step 5: Query Alerts

```
# Get all high-priority alerts for an exam
def get_high_alerts(exam_id):
    cursor.execute("""
        SELECT id, camera_id, timestamp, alert_type, seat, confidence
        FROM alerts
        WHERE exam_id = %s AND priority = 'high'
        ORDER BY timestamp DESC
    """, (exam_id,))

    alerts = cursor.fetchall()
    for alert in alerts:
        print(f"Alert {alert[0]}: Camera {alert[1]}, {alert[3]} at seat {alert[4]}, "
```

```
            f"confidence {alert[5]:.0%}")
    return alerts

# Get summary after exam
def get_exam_summary(exam_id):
    cursor.execute("""
        SELECT
            COUNT(*) as total,
            SUM(CASE WHEN status = 'confirmed' THEN 1 ELSE 0 END) as confirmed,
            SUM(CASE WHEN status = 'dismissed' THEN 1 ELSE 0 END) as dismissed,
            SUM(CASE WHEN status = 'pending' THEN 1 ELSE 0 END) as pending,
            AVG(confidence) as avg_confidence
        FROM alerts
        WHERE exam_id = %s
    """, (exam_id,))

    result = cursor.fetchone()
    print(f"Exam Summary:")
    print(f"  Total alerts: {result[0]}")
    print(f"  Confirmed: {result[1]}")
    print(f"  Dismissed (false alarms): {result[2]}")
    print(f"  Pending review: {result[3]}")
    print(f"  Average confidence: {result[4]:.0%}")
```

Using SQLAlchemy (Easier Approach)

SQLAlchemy lets you use Python objects instead of writing raw SQL.

```
from sqlalchemy import create_engine, Column, Integer, String, Float, DateTime
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from datetime import datetime

# Setup
engine = create_engine('postgresql://postgres:password@localhost/examguard')
Base = declarative_base()
Session = sessionmaker(bind=engine)

# Define Alert as a Python class
class Alert(Base):
    __tablename__ = 'alerts'

    id = Column(Integer, primary_key=True)
    exam_id = Column(Integer, nullable=False)
    camera_id = Column(Integer, nullable=False)
    timestamp = Column(DateTime, default=datetime.utcnow)
    alert_type = Column(String(50), nullable=False)
    seat = Column(String(10))
    confidence = Column(Float)
    priority = Column(String(10))
    evidence_path = Column(String(255))
    status = Column(String(20), default='pending')
```

```
# Create tables
Base.metadata.create_all(engine)

# Save an alert (much easier!)
session = Session()
new_alert = Alert(
    exam_id=1,
    camera_id=3,
    alert_type="phone_detected",
    seat="C2",
    confidence=0.92,
    priority="high",
    evidence_path="/evidence/alert_001.mp4"
)
session.add(new_alert)
session.commit()

# Query alerts (much easier!)
high_alerts = session.query(Alert).filter_by(
    exam_id=1,
    priority='high'
).all()

for alert in high_alerts:
    print(f"Alert {alert.id}: {alert.alert_type} at seat {alert.seat}")
```

Saving Evidence Video Clips

```
import cv2
import os
from datetime import datetime

def save_evidence_clip(frames, alert_id, output_dir="/evidence"):
    """Save suspicious frames as a video clip."""
    os.makedirs(output_dir, exist_ok=True)

    filename = f"alert_{alert_id}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.mp4"
    filepath = os.path.join(output_dir, filename)

    # Write frames to video file
    h, w = frames[0].shape[:2]
    writer = cv2.VideoWriter(
        filepath,
        cv2.VideoWriter_fourcc(*'mp4v'),
        15,  # fps
        (w, h)
    )

    for frame in frames:
        writer.write(frame)
    writer.release()
```

```
    print(f"Evidence saved: {filepath}")
    return filepath

# Usage: Keep a buffer of recent frames
from collections import deque
frame_buffer = deque(maxlen=90)  # Last 3 seconds at 30fps

while True:
    frame = camera.read()
    frame_buffer.append(frame)

    if ai_detects_something:
        # Save the last 3 seconds as evidence
        evidence_path = save_evidence_clip(list(frame_buffer), alert_id)
        save_alert(..., evidence_path=evidence_path)
```

What You Need to Learn

- 1. SQL basics — SELECT, INSERT, UPDATE, WHERE, JOIN, COUNT, AVG
- 2. Database design — How to organize tables and relationships
- 3. Python database connection — psycopg2 or SQLAlchemy
- 4. Indexing — Making queries fast (important for large datasets)
- 5. Backup — How to back up exam data safely

SQL Cheat Sheet (The Essentials)

```
-- Insert a new alert
INSERT INTO alerts (exam_id, camera_id, alert_type, seat, confidence)
VALUES (1, 3, 'phone_detected', 'C2', 0.92);

-- Get all alerts for an exam
SELECT * FROM alerts WHERE exam_id = 1;

-- Get high-priority alerts
SELECT * FROM alerts WHERE priority = 'high' ORDER BY timestamp DESC;

-- Update alert status
UPDATE alerts SET status = 'confirmed' WHERE id = 1;

-- Count alerts by type
SELECT alert_type, COUNT(*) FROM alerts GROUP BY alert_type;

-- Get average confidence by camera
SELECT camera_id, AVG(confidence) FROM alerts GROUP BY camera_id;

-- Get alerts with invigilator actions (JOIN)
SELECT a.*, ia.action, ia.notes
FROM alerts a
LEFT JOIN invigilator_actions ia ON a.id = ia.alert_id
WHERE a.exam_id = 1;
```

Mini Project: Alert Logging System

Goal

Build a simple system that stores alerts and lets you query them.

Step 1: Use SQLite (No Installation Needed)

```
import sqlite3
from datetime import datetime

# SQLite creates a file — no server needed!
conn = sqlite3.connect('examguard.db')
cursor = conn.cursor()

# Create table
cursor.execute("""
  CREATE TABLE IF NOT EXISTS alerts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    camera_id INTEGER,
    timestamp TEXT,
    alert_type TEXT,
    seat TEXT,
    confidence REAL,
    priority TEXT,
    status TEXT DEFAULT 'pending'
  )
""")
conn.commit()
```

Step 2: Add Some Alerts

```
import random

alert_types = ['phone_detected', 'sustained_gaze', 'anomaly', 'note_passing']
priorities = ['high', 'medium', 'low']
seats = ['A1', 'A2', 'B1', 'B2', 'C1', 'C2', 'D1', 'D2']

# Insert 50 fake alerts
for i in range(50):
  cursor.execute("""
    INSERT INTO alerts (camera_id, timestamp, alert_type, seat, confidence, priority)
    VALUES (?, ?, ?, ?, ?, ?)
  """, (
    random.randint(1, 4),
    datetime.now().isoformat(),
    random.choice(alert_types),
    random.choice(seats),
    round(random.uniform(0.5, 0.99), 2),
    random.choice(priorities)
  ))

conn.commit()
print("50 alerts added!")
```

Step 3: Query the Data

```
# How many alerts total?
cursor.execute("SELECT COUNT(*) FROM alerts")
print(f"Total alerts: {cursor.fetchone()[0]}")

# Alerts by type
cursor.execute("SELECT alert_type, COUNT(*) FROM alerts GROUP BY alert_type")
for row in cursor.fetchall():
    print(f"  {row[0]}: {row[1]}")

# High priority alerts
cursor.execute("SELECT * FROM alerts WHERE priority = 'high' ORDER BY confidence DESC")
print("\nHigh priority alerts:")
for row in cursor.fetchall():
    print(f"  Camera {row[1]}, Seat {row[4]}, Confidence {row[5]}, Type: {row[3]}")

# Average confidence by camera
cursor.execute("SELECT camera_id, AVG(confidence), COUNT(*) FROM alerts GROUP BY camera_id")
print("\nCamera statistics:")
for row in cursor.fetchall():
    print(f"  Camera {row[0]}: Avg confidence {row[1]:.2f}, Total alerts: {row[2]}")
```

Key Takeaways

- 1. The database is ExamGuard's memory — without it, every alert disappears after the exam
- 2. PostgreSQL for production, SQLite for learning — start with SQLite, upgrade later
- 3. Store everything — alerts, actions, evidence paths, timestamps, confidence scores
- 4. Evidence clips go to file storage — database stores the path, not the video file
- 5. SQL is a must-learn skill — a few basic commands cover 90% of what you need
- 6. Good database design now saves pain later — plan your tables before building

REST API — How Parts of the System Talk to Each Other

What Is This?

ExamGuard is not ONE program. It is MANY programs working together:

```
Camera Reader → sends frames to → AI Engine
AI Engine → sends alerts to → Dashboard
Dashboard → sends actions to → Database
Database → sends history to → Report Generator
```

How do these separate programs communicate? Through an API (Application Programming Interface).

A REST API is a standard way for programs to send and receive data over a network using simple HTTP requests — the same technology your web browser uses.

How Does It Work? (Simple Explanation)

Think of a REST API like a waiter in a restaurant:

```
You (the client)    → "I want a pizza"    → Waiter (API)
Waiter (API)        → Takes order to      → Kitchen (server)
Kitchen (server)    → Makes pizza         → Waiter (API)
Waiter (API)        → Delivers pizza      → You (the client)
```

In code:

```
Dashboard (client)  → GET /api/alerts     → Server (API)
Server (API)        → Queries database    → Database
Database            → Returns data        → Server (API)
Server (API)        → Sends JSON response → Dashboard (client)
```

The Four Main Actions (HTTP Methods)

```
GET    = Read data      "Show me all alerts"
POST   = Create data    "Add a new alert"
PUT    = Update data    "Change alert status to confirmed"
DELETE = Remove data    "Delete this old alert"
```

What Is JSON?

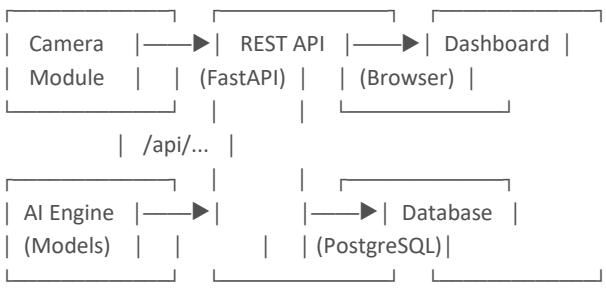
JSON is the FORMAT that data travels in. It looks like a Python dictionary:

```
{
  "id": 1,
  "camera_id": 3,
  "alert_type": "phone_detected",
  "seat": "C2",
  "confidence": 0.92,
  "priority": "high",
  "status": "pending"
}
```

Every API sends and receives data in JSON format.

WHY ExamGuard Needs APIs

The Architecture



Why Not Just One Big Program?

One big program:
- If AI engine crashes → EVERYTHING crashes
- If you update the dashboard → must restart AI engine too
- Cannot scale (run more AI engines for more cameras)
- Hard to develop (one person's change breaks another's code)

Separate programs + API:
- If AI engine crashes → dashboard still works, shows "AI offline"
- Update dashboard → AI engine keeps running, does not notice
- Scale: run 3 AI engines behind one API for more cameras
- Develop: different people work on different parts independently

ExamGuard API Endpoints

Camera Endpoints

GET /api/cameras → List all cameras and their status
GET /api/cameras/3 → Get details of camera 3
POST /api/cameras/3/snapshot → Get current frame from camera 3

Alert Endpoints

GET /api/alerts → List all alerts (with filters)
GET /api/alerts?priority=high → List only high-priority alerts
GET /api/alerts/42 → Get details of alert 42
POST /api/alerts → Create a new alert (AI engine sends this)
PUT /api/alerts/42 → Update alert 42 (change status)

Action Endpoints

POST /api/alerts/42/confirm → Invigilator confirms this alert
POST /api/alerts/42/dismiss → Invigilator dismisses this alert

Statistics Endpoints

GET /api/stats/exam/1 → Get statistics for exam 1
GET /api/stats/camera/3 → Get statistics for camera 3

Frame Processing Endpoint

POST /api/process → Send an image, get AI analysis back

Building with FastAPI

Why FastAPI?

Flask: Simple, widely used, synchronous
FastAPI: Modern, FAST, async, automatic documentation, type checking

For ExamGuard: FastAPI is better because:
1. It is faster (important for real-time)
2. Auto-generates API documentation
3. Built-in data validation
4. Easy to learn if you know Flask

Step 1: Install

pip install fastapi uvicorn python-multipart

Step 2: Basic API

```
from fastapi import FastAPI
from pydantic import BaseModel
from datetime import datetime
from typing import Optional, List

app = FastAPI(title="ExamGuard API", version="1.0")

# —— Data Models ——

class AlertCreate(BaseModel):
    camera_id: int
    alert_type: str
    seat: str
    confidence: float
    priority: str  # "high", "medium", "low"

class Alert(BaseModel):
    id: int
    camera_id: int
    alert_type: str
    seat: str
    confidence: float
    priority: str
    status: str
    timestamp: str

# —— In-memory storage (use database in production) ——

alerts_db = []
alert_counter = 0

# —— Endpoints ——

@app.get("/")
```

```
def root():
    return {"message": "ExamGuard API is running"}

@app.get("/api/alerts", response_model=List[Alert])
def get_alerts(priority: Optional[str] = None):
    """Get all alerts, optionally filtered by priority."""
    if priority:
        return [a for a in alerts_db if a["priority"] == priority]
    return alerts_db

@app.get("/api/alerts/{alert_id}")
def get_alert(alert_id: int):
    """Get a specific alert by ID."""
    for alert in alerts_db:
        if alert["id"] == alert_id:
            return alert
    return {"error": "Alert not found"}

@app.post("/api/alerts")
def create_alert(alert: AlertCreate):
    """Create a new alert (called by AI engine)."""
    global alert_counter
    alert_counter += 1

    new_alert = {
        "id": alert_counter,
        "camera_id": alert.camera_id,
        "alert_type": alert.alert_type,
        "seat": alert.seat,
        "confidence": alert.confidence,
        "priority": alert.priority,
        "status": "pending",
        "timestamp": datetime.now().isoformat()
    }
    alerts_db.append(new_alert)
    return new_alert

@app.post("/api/alerts/{alert_id}/confirm")
def confirm_alert(alert_id: int):
    """Invigilator confirms this alert as real cheating."""
    for alert in alerts_db:
        if alert["id"] == alert_id:
            alert["status"] = "confirmed"
            return {"message": f"Alert {alert_id} confirmed"}
    return {"error": "Alert not found"}

@app.post("/api/alerts/{alert_id}/dismiss")
def dismiss_alert(alert_id: int):
    """Invigilator dismisses this alert as false alarm."""
    for alert in alerts_db:
        if alert["id"] == alert_id:
```

```
            alert["status"] = "dismissed"
            return {"message": f"Alert {alert_id} dismissed"}
    return {"error": "Alert not found"}

@app.get("/api/stats")
def get_stats():
    """Get overall statistics."""
    total = len(alerts_db)
    confirmed = sum(1 for a in alerts_db if a["status"] == "confirmed")
    dismissed = sum(1 for a in alerts_db if a["status"] == "dismissed")
    pending = sum(1 for a in alerts_db if a["status"] == "pending")

    return {
        "total_alerts": total,
        "confirmed": confirmed,
        "dismissed": dismissed,
        "pending": pending,
        "false_alarm_rate": dismissed / total if total > 0 else 0
    }
```

Step 3: Run the API

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Step 4: Test with Browser

Open `http://localhost:8000/docs` — FastAPI auto-generates interactive documentation where you can test every endpoint!

Image Processing Endpoint

The most important endpoint — send an image, get AI analysis back.

```
from fastapi import FastAPI, UploadFile, File
import cv2
import numpy as np
from ultralytics import YOLO

app = FastAPI()
model = YOLO("yolov8n.pt")

@app.post("/api/process")
async def process_frame(file: UploadFile = File(...)):
    """
    Send an image, get AI analysis back.
    This is what edge devices call.
    """
    # Read the uploaded image
    contents = await file.read()
    nparr = np.frombuffer(contents, np.uint8)
    frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    # Run YOLO detection
    results = model(frame, verbose=False)
```

```
    # Format results
    detections = []
    for box in results[0].boxes:
        detections.append({
            "class": results[0].names[int(box.cls)],
            "confidence": float(box.conf),
            "bbox": box.xyxy[0].tolist()
        })

    # Determine if suspicious
    suspicious_items = ["cell phone", "book"]
    is_suspicious = any(d["class"] in suspicious_items for d in detections)

    return {
        "is_suspicious": is_suspicious,
        "detections": detections,
        "total_objects": len(detections)
    }
```

Calling the API from Python (Client Side)

```
import requests

# Send an image to the API
with open("test_image.jpg", "rb") as f:
    response = requests.post(
        "http://localhost:8000/api/process",
        files={"file": f}
    )

result = response.json()
print(f"Suspicious: {result['is_suspicious']}")
print(f"Detections: {result['detections']}")
```

What You Need to Learn

- 1. REST concepts — GET, POST, PUT, DELETE, JSON, status codes
- 2. FastAPI basics — routes, request/response models, file uploads
- 3. HTTP status codes — 200 (OK), 201 (Created), 404 (Not Found), 500 (Server Error)
- 4. Testing APIs — using the browser, curl, or Python requests library
- 5. API design — naming conventions, structuring endpoints logically

Mini Project: Image Classification API

Goal

Build an API that accepts an image and returns whether it shows "cheating" or "normal" behavior.

Step 1: Create the API

```
# save as app.py
from fastapi import FastAPI, UploadFile, File
```

```
from fastapi.responses import HTMLResponse
import cv2
import numpy as np
from ultralytics import YOLO

app = FastAPI(title="ExamGuard Classifier API")
model = YOLO("yolov8n.pt")

# Suspicious objects
SUSPICIOUS = {"cell phone", "book", "laptop"}

@app.get("/", response_class=HTMLResponse)
def home():
    return """
    <h1>ExamGuard Classifier API</h1>
    <form action="/api/classify" method="post" enctype="multipart/form-data">
        <input type="file" name="image" accept="image/*">
        <button type="submit">Analyze</button>
    </form>
    """

@app.post("/api/classify")
async def classify_image(image: UploadFile = File(...)):
    # Read image
    contents = await image.read()
    nparr = np.frombuffer(contents, np.uint8)
    frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    # Run detection
    results = model(frame, verbose=False)

    detections = []
    suspicious_found = []

    for box in results[0].boxes:
        class_name = results[0].names[int(box.cls)]
        conf = float(box.conf)
        detections.append({"object": class_name, "confidence": round(conf, 2)})

        if class_name in SUSPICIOUS and conf > 0.5:
            suspicious_found.append(class_name)

    verdict = "SUSPICIOUS" if suspicious_found else "NORMAL"

    return {
        "verdict": verdict,
        "suspicious_objects": suspicious_found,
        "all_detections": detections,
        "total_objects": len(detections)
    }
```

```
# Run: uvicorn app:app --reload --port 8000
# Then open http://localhost:8000 in your browser
```

Step 2: Run It

```
uvicorn app:app --reload --port 8000
```

Step 3: Test It

- Open http://localhost:8000 in your browser
- Upload a photo
- See the classification result
- Also try http://localhost:8000/docs for the auto-generated API docs

Key Takeaways

- 1. APIs let separate programs communicate — camera module, AI engine, dashboard, database all talk through APIs
- 2. REST uses HTTP — the same technology as web browsing, simple and universal
- 3. FastAPI is the best choice for Python AI projects — fast, easy, auto-documentation
- 4. JSON is the data format — everything is sent and received as JSON
- 5. The /api/process endpoint is ExamGuard's core — send a frame in, get analysis back
- 6. Start with simple endpoints — get them working, then add complexity

Model Deployment — From Jupyter Notebook to Production System

What Is This?

You trained a model in a Jupyter notebook. It works great. You are proud.

But here is the problem:

Jupyter Notebook:
- YOU click "Run" manually
- Runs on YOUR laptop
- Stops when you close the lid
- No error handling
- No monitoring
- No auto-restart

Real Exam:
- Must start automatically when exam begins
- Runs on a SERVER (not your laptop)
- Must run for 3+ hours without stopping
- Must handle errors gracefully
- Must be monitored (is it still working?)
- Must auto-restart if it crashes

Model deployment means packaging your model into a reliable system that runs without you babysitting it.

WHY ExamGuard Needs This

The Nightmare Scenario

Exam starts at 2:00 PM.
200 students sit down.
ExamGuard is running on your laptop in a Jupyter notebook.

2:15 PM: Your laptop goes to sleep. System stops.
2:20 PM: You realize and wake it up. Restart the notebook.
2:22 PM: Python crashes — "CUDA out of memory"
2:30 PM: You restart, but the camera connection code has a bug.
2:45 PM: Finally working again.

Result: 45 minutes of exam with NO monitoring.
Any cheating in that window? Completely missed.

What Deployment Solves

Exam starts at 2:00 PM.
ExamGuard runs on a dedicated server, in Docker containers.

2:00 PM: System auto-starts. All cameras connected.
2:15 PM: Camera 3 disconnects. System logs error, reconnects in 5 seconds.
2:30 PM: AI engine uses too much memory. Auto-restart in 3 seconds.
2:31 PM: AI engine back online. No alerts lost (queued during restart).
5:00 PM: Exam ends. System generates report automatically.

Result: 3 hours of continuous monitoring. Every second covered.

Key Concepts

1. Docker — Package Everything Into a Box

What: Docker creates a "container" — a mini computer inside your computer that has EVERYTHING your code needs.

Why: "It works on my machine" problem. Your code needs specific versions of Python, PyTorch, CUDA, OpenCV, etc. Docker packages ALL of this together.

Without Docker:
"Install Python 3.10, then PyTorch 2.0, then CUDA 11.8, then OpenCV 4.8..."
"Oh, it does not work on your computer because you have Python 3.8"

With Docker:
"Run this one command: docker run examguard"
Everything is already inside the container. Works on ANY computer.

2. Model Saving and Loading

```
import torch

# —— Save the model after training ——
model = YourTrainedModel()
# ... training code ...

# Save the entire model
torch.save(model.state_dict(), 'examguard_model.pth')
print("Model saved!")

# —— Load in production ——
model = YourModelClass()  # Create empty model
model.load_state_dict(torch.load('examguard_model.pth'))
model.eval()  # Switch to inference mode (no training)
print("Model loaded and ready!")
```

3. Health Checks

```
# The server needs to know: "Is ExamGuard still working?"

from fastapi import FastAPI
import torch
import psutil

app = FastAPI()

@app.get("/health")
def health_check():
    """Called every 30 seconds by the monitoring system."""
    checks = {
        "api": "ok",
        "gpu": "ok" if torch.cuda.is_available() else "error",
```

```
        "gpu_memory_used": f"{torch.cuda.memory_allocated()/1e9:.1f}GB"
            if torch.cuda.is_available() else "N/A",
        "cpu_percent": psutil.cpu_percent(),
        "ram_percent": psutil.virtual_memory().percent,
        "cameras_connected": check_cameras(),
        "model_loaded": model is not None
    }

    all_ok = all(v != "error" for v in checks.values() if isinstance(v, str))
    return {"status": "healthy" if all_ok else "unhealthy", "checks": checks}
```

4. Logging

```
import logging
from datetime import datetime

# Setup logging
logging.basicConfig(
    filename=f'examguard_{datetime.now().strftime("%Y%m%d")}.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

logger = logging.getLogger('examguard')

# Use throughout your code
logger.info("ExamGuard started")
logger.info(f"Connected to {len(cameras)} cameras")
logger.warning("Camera 3 disconnected, attempting reconnect...")
logger.error("AI model failed to load!")
logger.info("Alert #42: Phone detected at seat C2, confidence 0.92")

# Log file looks like:
# 2026-03-15 14:00:01 - INFO - ExamGuard started
# 2026-03-15 14:00:03 - INFO - Connected to 4 cameras
# 2026-03-15 14:15:22 - WARNING - Camera 3 disconnected, attempting reconnect...
# 2026-03-15 14:15:27 - INFO - Camera 3 reconnected
```

Docker: Step by Step

Step 1: Create a Dockerfile

```
# Dockerfile
# This tells Docker how to build your container

# Start with a base image that has Python and CUDA
FROM nvidia/cuda:11.8.0-runtime-ubuntu22.04

# Install Python
RUN apt-get update && apt-get install -y python3 python3-pip
RUN apt-get install -y libgl1-mesa-glx libglib2.0-0  # For OpenCV

# Set working directory
```

```
WORKDIR /app

# Copy requirements and install
COPY requirements.txt .
RUN pip3 install -r requirements.txt

# Copy your code and models
COPY . .

# Expose the API port
EXPOSE 8000

# Command to run when container starts
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 2: Create requirements.txt

```
fastapi==0.104.0
uvicorn==0.24.0
torch==2.1.0
torchvision==0.16.0
ultralytics==8.0.200
opencv-python-headless==4.8.1.78
psycopg2-binary==2.9.9
numpy==1.26.0
```

Step 3: Build and Run

```
# Build the Docker image
docker build -t examguard .

# Run it
docker run -d \
  --name examguard-api \
  --gpus all \
  -p 8000:8000 \
  -v /data/evidence:/app/evidence \
  --restart unless-stopped \
  examguard

# Flags explained:
# -d              : Run in background
# --name          : Give it a name
# --gpus all      : Use GPU
# -p 8000:8000    : Expose port 8000
# -v /data:/app/data : Share a folder for evidence storage
# --restart unless-stopped : Auto-restart if it crashes!
```

Step 4: Docker Compose (Multiple Services)

For the full ExamGuard system with multiple components:

```
# docker-compose.yml
version: '3.8'
```

```yaml
services:
  # AI Engine
  ai-engine:
    build: ./ai_engine
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]
    ports:
      - "8001:8000"
    restart: unless-stopped
    depends_on:
      - database
      - redis

  # API Server
  api-server:
    build: ./api_server
    ports:
      - "8000:8000"
    restart: unless-stopped
    depends_on:
      - database

  # Dashboard
  dashboard:
    build: ./dashboard
    ports:
      - "3000:3000"
    restart: unless-stopped

  # Database
  database:
    image: postgres:15
    environment:
      POSTGRES_DB: examguard
      POSTGRES_PASSWORD: secure_password
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: unless-stopped

  # Redis (real-time cache)
  redis:
    image: redis:7
    restart: unless-stopped

volumes:
  pgdata:
```

```
# Start EVERYTHING with one command:
docker-compose up -d

# Stop everything:
docker-compose down

# See logs:
docker-compose logs -f ai-engine
```

Production Checklist

Before deploying ExamGuard for a real exam:

[ ] Models saved and tested (.pth or .engine files)
[ ] Docker containers built and tested
[ ] Health check endpoint working
[ ] Logging configured (file + console)
[ ] Auto-restart on crash (--restart unless-stopped)
[ ] Camera connections tested with real cameras
[ ] Database backed up before exam
[ ] Evidence storage has enough disk space
[ ] GPU memory checked (no leaks)
[ ] API endpoints all tested
[ ] Dashboard loads correctly
[ ] Network connectivity verified
[ ] Fallback plan if system goes down (human invigilators ready)

What You Need to Learn

- 1. Docker basics — images, containers, Dockerfile, docker-compose
- 2. Model serialization — saving and loading PyTorch models
- 3. Logging — Python logging module, log levels, log files
- 4. Health checks — monitoring system status
- 5. Process management — auto-restart, resource limits, graceful shutdown

Mini Project: Package YOLO in Docker

Goal

Create a Docker container that runs YOLO as an API service.

Step 1: Create Project Structure

```
yolo_service/
├── Dockerfile
├── requirements.txt
├── main.py
└── yolov8n.pt      (download this first)
```

Step 2: main.py

```
from fastapi import FastAPI, UploadFile, File
from ultralytics import YOLO
import cv2
import numpy as np
```

```
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("yolo-service")

app = FastAPI(title="YOLO Detection Service")

# Load model once at startup
logger.info("Loading YOLO model...")
model = YOLO("yolov8n.pt")
logger.info("Model loaded!")

@app.get("/health")
def health():
    return {"status": "healthy", "model": "yolov8n"}

@app.post("/detect")
async def detect(image: UploadFile = File(...)):
    contents = await image.read()
    nparr = np.frombuffer(contents, np.uint8)
    frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    results = model(frame, verbose=False)

    detections = []
    for box in results[0].boxes:
        detections.append({
            "class": results[0].names[int(box.cls)],
            "confidence": round(float(box.conf), 3),
        })

    logger.info(f"Detected {len(detections)} objects")
    return {"detections": detections}
```

Step 3: Dockerfile

```
FROM python:3.10-slim
RUN apt-get update && apt-get install -y libgl1-mesa-glx libglib2.0-0
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 4: Build and Run

```
docker build -t yolo-service .
docker run -d -p 8000:8000 --restart unless-stopped --name yolo yolo-service
```

Step 5: Test

```
# Check health
curl http://localhost:8000/health
```

```
# Send an image for detection
curl -X POST http://localhost:8000/detect -F "image=@test_photo.jpg"
```

Key Takeaways

- 1. Jupyter is for development, Docker is for production — never run Jupyter in a real exam
- 2. Docker packages everything — Python, libraries, models, code, all in one container
- 3. Auto-restart is essential — crashes WILL happen, recovery must be automatic
- 4. Logging saves you — when something goes wrong at 2 AM, logs tell you what happened
- 5. Health checks keep you informed — monitor system status continuously
- 6. docker-compose runs the whole system — one command starts everything

Phase 7: System Building — Learning Resources

Camera Streams & RTSP

YouTube Videos

- "RTSP Camera Stream with Python OpenCV" by Murtaza's Workshop — hands-on tutorial
- "IP Camera Streaming Python" by Tech with Tim — step-by-step
- "OpenCV VideoCapture Tutorial" by ProgrammingKnowledge — covers all input sources
- "Multi-camera OpenCV Python" by Nicholas Renotte — reading multiple feeds

Practice

- Use your phone as an IP camera (IP Webcam app for Android, EpocCam for iPhone)
- Connect to any RTSP stream and display it with OpenCV
- Build a multi-camera display grid

Edge Computing

YouTube Videos

- "Getting Started with NVIDIA Jetson Nano" by JetsonHacks — the essential Jetson tutorial
- "Run YOLO on Jetson Nano" by NVIDIA Developer — official guide
- "Edge AI Explained" by Deloitte — understand the concept
- "Raspberry Pi AI Projects" by ExplainingComputers — budget edge computing

Documentation

- NVIDIA Jetson Developer Kit: developer.nvidia.com/embedded/jetson-developer-kits
- Google Coral Edge TPU: coral.ai/docs
- PyTorch Mobile: pytorch.org/mobile

Practice

- If you have a Jetson: Run YOLOv8-nano on it
- If not: Simulate edge computing by comparing YOLOv8-nano vs YOLOv8-large speeds
- Try model export to TensorRT or ONNX

GPU Programming

YouTube Videos

- "CUDA Programming for Beginners" by CoffeeBeforeArch — gentle introduction
- "PyTorch GPU Tutorial" by sentdex — practical GPU usage in PyTorch
- "NVIDIA CUDA Explained" by Fireship — quick overview
- "Mixed Precision Training" by NVIDIA Developer — speed up training 2x

Documentation

- PyTorch CUDA guide: pytorch.org/docs/stable/cuda.html
- NVIDIA CUDA Toolkit: developer.nvidia.com/cuda-toolkit

- cuDNN: developer.nvidia.com/cudnn

Practice

- Run CPU vs GPU benchmark on any model
- Try mixed precision training
- Monitor GPU memory usage during training

Web Dashboard

YouTube Videos

- "Flask Tutorial - Full Course for Beginners" by Tech with Tim — complete Flask guide
- "FastAPI Tutorial" by Traversy Media — modern Python web framework
- "Build a Dashboard with Flask" by Pretty Printed — directly applicable
- "WebSocket Tutorial" by Fireship — real-time communication
- "HTML/CSS Crash Course" by Traversy Media — just enough web skills

Courses

- "Flask Web Development" on Udemy (by Jose Portilla) — comprehensive
- freeCodeCamp Responsive Web Design certification — free, covers HTML/CSS

Practice

- Build a simple Flask app that shows webcam feed in browser
- Add fake alerts that appear in real-time
- Style it with CSS to look like a monitoring dashboard

Database

YouTube Videos

- "SQL Tutorial - Full Database Course" by freeCodeCamp — complete SQL from scratch
- "PostgreSQL Tutorial for Beginners" by Amigoscode — setup and basic queries
- "SQLAlchemy Tutorial" by Pretty Printed — Python + database made easy
- "Redis Crash Course" by Traversy Media — real-time caching

Practice

- Start with SQLite (no installation needed)
- Build the alert logging system from the mini project
- Learn these SQL commands: SELECT, INSERT, UPDATE, WHERE, JOIN, GROUP BY, COUNT, AVG
- Move to PostgreSQL when ready for production

REST API

YouTube Videos

- "FastAPI Course for Beginners" by freeCodeCamp — 4-hour comprehensive course
- "REST API Concepts" by WebDevSimplified — understand the theory

- "Build a REST API with FastAPI" by Traversy Media — practical tutorial
- "Postman Tutorial" by Traversy Media — testing APIs

Tools

- Postman — GUI tool for testing APIs (download from postman.com)
- curl — command-line tool for testing APIs (built into most systems)
- FastAPI /docs — auto-generated interactive documentation

Practice

- Build a simple CRUD API (Create, Read, Update, Delete)
- Add file upload endpoint (for images)
- Test with Postman or the auto-generated docs page

Docker & Deployment

YouTube Videos

- "Docker Tutorial for Beginners" by TechWorld with Nana — best Docker intro
- "Docker for Data Scientists" by Data School — directly applicable
- "Docker Compose Tutorial" by NetworkChuck — running multiple services
- "Deploying ML Models" by Krish Naik — model deployment specifically

Documentation

- Docker official tutorial: docs.docker.com/get-started
- Docker Hub: hub.docker.com (pre-built images)

Practice

- Install Docker Desktop
- Build a Docker image for a simple Python script
- Package your YOLO API in Docker
- Use docker-compose to run API + database together

Recommended Learning Order

Week 1-2: Camera streams → Phone camera mini project
Week 3-4: Flask dashboard → Webcam + fake alerts mini project
Week 5-6: Database (SQLite) → Alert logging mini project
Week 7-8: REST API (FastAPI) → Image classification API mini project
Week 9-10: Docker → Package YOLO in Docker mini project
Week 11-12: GPU programming → CPU vs GPU benchmark
Week 13-14: Edge computing → Run YOLO on lightweight setup

Quick Tip

Build each component STANDALONE first. Get it working independently. THEN start connecting them together. Trying to build everything at once is a recipe for confusion.

Phase 1: Camera reading (standalone) ✓
Phase 2: AI detection (standalone) ✓

Phase 3: Dashboard (standalone, fake data) ✓
Phase 4: Database (standalone) ✓
Phase 5: API (standalone) ✓
Phase 6: Camera → API → AI → Database → Dashboard (INTEGRATED)

Tools to Install

```
# Web framework
pip install flask flask-socketio
pip install fastapi uvicorn python-multipart

# Database
pip install psycopg2-binary sqlalchemy
# Also install PostgreSQL from postgresql.org

# Docker
# Download Docker Desktop from docker.com

# Utilities
pip install requests    # For testing APIs
pip install psutil      # For system monitoring
pip install redis       # For Redis cache (optional)
```